

Library Management System

Database Architecture & Design Specification

1. Name and Purpose of the Application

Application Name

LibSphere — Library Management System

Purpose

LibSphere is designed to digitize and automate core library operations. The primary goals of the system are:

- **Inventory Management:** Maintain accurate records of books, catalog categories, and author details.
- **Member Management:** Track registered library members, membership statuses, and contact information.
- **Circulation Tracking:** Monitor book checkouts, due dates, returns, and track active loans.
- **Operational Efficiency:** Eliminate manual register keeping, reduce inventory loss, and maintain data integrity across all transactions.

2. Required Tables

To effectively manage library operations, the database consists of six core relational tables:

1. **Categories:** Classifies titles into genres or subjects (e.g., Fiction, Computer Science, History).
2. **Authors:** Stores information about individuals who wrote the cataloged books.

- 3. **Books**: Catalog of book titles along with publication details and inventory counts.
- 4. **Book_Authors**: A bridge table enabling a many-to-many relationship between books and authors.
- 5. **Members**: Stores library patron records, contact information, and membership status.
- 6. **Loans**: Records borrowing transactions, return dates, and checkout statuses.

3. Columns / Attributes for Each Table

Categories Table

Stores classification records for grouping books by topic or genre.

Column Name	Data Type	Constraints	D
category_id	INT	PRIMARY KEY, AUTO_INCREMENT	U c
category_name	VARCHAR(50)	NOT NULL, UNIQUE	N d

Authors Table

Contains biographic information for authors represented in the catalog.

Column Name	Data Type	Constraints	D
author_id	INT	PRIMARY KEY, AUTO_INCREMENT	U a
first_name	VARCHAR(50)	NOT NULL	A
last_name	VARCHAR(50)	NOT NULL	A

Column Name	Data Type	Constraints	D
bio	TEXT	NULLABLE	B

Books Table

Holds metadata and inventory tracking numbers for book titles.

Column Name	Data Type	Constraints	D
book_id	INT	PRIMARY KEY, AUTO_INCREMENT	U b
title	VARCHAR(150)	NOT NULL	F
isbn	VARCHAR(13)	NOT NULL, UNIQUE	In N
category_id	INT	FOREIGN KEY	R C)
publication_year	INT	NOT NULL	Y p
total_copies	INT	NOT NULL, DEFAULT 1	T o
available_copies	INT	NOT NULL, DEFAULT 1	C f

Book_Authors Table

Associates authors with books to support titles written by multiple authors.

Column Name	Data Type	Constraints	Description
book_id	INT	PRIMARY KEY, FOREIGN KEY	Reference to book catalog
author_id	INT	PRIMARY KEY, FOREIGN KEY	Reference to author information

Members Table

Stores profile information for registered library patrons.

Column Name	Data Type	Constraints	Description
member_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each member
first_name	VARCHAR(50)	NOT NULL	Member's first name
last_name	VARCHAR(50)	NOT NULL	Member's last name
email	VARCHAR(100)	NOT NULL, UNIQUE	Member's email address
phone	VARCHAR(20)	NOT NULL	Member's phone number
membership_date	DATE	NOT NULL	Date when membership was created
status	VARCHAR(20)	NOT NULL, DEFAULT 'Active'	Current membership status

Loans Table

Tracks individual borrowing events and return records.

Column Name	Data Type	Constraints	
loan_id	INT	PRIMARY KEY, AUTO_INCREMENT	U ic
book_id	INT	FOREIGN KEY	R B
member_id	INT	FOREIGN KEY	R M
loan_date	DATE	NOT NULL	D b
due_date	DATE	NOT NULL	S
return_date	DATE	NULLABLE	A re
status	VARCHAR(20)	NOT NULL, DEFAULT 'Issued'	L R

4. Relationships Between Tables

- **Categories to Books (One-to-Many):** One category can contain multiple books, but each book belongs to exactly one category. Linked via `Books.category_id`.
- **Books to Authors (Many-to-Many):** A single book can be written by multiple authors, and an author can write multiple books. This is resolved using the `Book_Authors` bridge table.
- **Members to Loans (One-to-Many):** A single library member can generate multiple loan records over time, but each loan record belongs to only one member. Linked via `Loans.member_id`.

- **Books to Loans (One-to-Many):** A single book title can be checked out multiple times over its lifecycle, but each specific loan transaction references only one book. Linked via `Loans.book_id`.

5. Overall Explanation

The **LibSphere** relational database schema is structured around Third Normal Form (3NF) principles to minimize data redundancy and enforce relational integrity:

1. **Normalized Entity Structure:** Separating authors, categories, and inventory items ensures that updating an author's name or a category title requires modifying only a single record without inconsistent duplicates.
2. **Flexible Author Attribution:** The `Book_Authors` junction table provides full flexibility for single-author works, co-authored textbooks, or anthology collections without altering the underlying `Books` table structure.
3. **Inventory & Circulation Control:** The `available_copies` field in `Books` acts as an operational check for loan transactions. When a new record is created in `Loans`, application logic decrements `available_copies`; upon population of `return_date`, the available count is incremented.
4. **Data Integrity Constraints:** Primary and foreign keys strictly govern foreign dependency constraints. Cascade protections prevent accidental deletion of registered members or book titles while active loans exist.